Probeklausur: Echtzeitsysteme (EZS)

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung & Erwartungshorizont

Allgemeine Hinweise

- **Zweck:** Vorbereitung auf die Papierklausur
- **Bearbeitungszeit:** 90 Minuten
- **Hilfsmittel:** Keine (Closed Book)
- **Maximale Punktzahl:** 100 Punkte

Alle Angaben sind Lehrannahmen. Hardware und Programmierwerkzeuge werden nicht benötigt. Zeigen Sie Rechenwege und begründen Sie Aussagen. API-Namen im Code sind vorgegeben; beurteilt wird ihr Einsatz, nicht die Erinnerung an Werkzeugmenüs.

Aufgabe 1: Scheduling-Theorie & RMS (25 Punkte)

Ein neues Steuergerät soll drei periodische Tasks im präemptiven Multitasking verarbeiten. Für die Tasks wurden folgende Parameter (Worst-Case Execution Time C_i, Periode T_i) ermittelt. Es gelten implizite Deadlines ($D_i = T_i$). Das Lehrmodell verwendet genau einen Prozessor, unabhängige periodische Aufgaben, vollständige Präemption und keine zusätzlichen Interrupt-, Wechsel- oder Sperrkosten. Die angegebenen C_i sind für dieses Modell als sichere obere Schranken vorgegeben, keine aus einer endlichen Messreihe garantierten Werte.

- τ_1: $C_1 = 2\,\text{ms}$, $T_1 = 8\,\text{ms}$
- τ_2: $C_2 = 3\,\text{ms}$, $T_2 = 12\,\text{ms}$
- τ_3: $C_3 = 4\,\text{ms}$, $T_3 = 24\,\text{ms}$

1.1 Berechnen Sie die Gesamt-CPU-Auslastung U des Systems. Zeigen Sie Ihren Rechenweg. *(5 Punkte)*

1.2 Vergeben Sie die Prioritäten der Tasks nach den Regeln des *Rate Monotonic Scheduling (RMS)*. Begründen Sie Ihre Wahl in einem Satz. *(5 Punkte)*

1.3 Prüfen Sie mithilfe der Liu-und-Layland-Schranke, ob dieses Task-Set garantiert schedulbar ist. Die Schranke für $n = 3$ lautet $\approx 78{,}0\,\%$. *(5 Punkte)*

1.4 Nennen Sie zwei Voraussetzungen des hier benutzten Liu-und-Layland-Tests. Erläutern Sie zu jeder einen konkreten Systemfall, in dem die einfache Rechnung unverändert nicht genügt. *(10 Punkte)*

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 1

1.1 Auslastung U:

$$U = \sum_{i=1}^{3} \frac{C_i}{T_i} = \frac{2}{8} + \frac{3}{12} + \frac{4}{24} = \frac{1}{4} + \frac{1}{4} + \frac{1}{6} = \frac{3}{12} + \frac{3}{12} + \frac{2}{12} = \frac{8}{12} = \frac{2}{3} \approx 0{,}667$$

Die Gesamt-CPU-Auslastung beträgt $U = \frac{2}{3}$ (ca. **66,7 %**). *(5 Punkte)*

1.2 RMS-Prioritäten: Regel: Je kürzer die Periode, desto höher die Priorität (*Rate Monotonic*).

- **Höchste Priorität:** τ_1 ($T_1 = 8\,\text{ms}$)
- **Mittlere Priorität:** τ_2 ($T_2 = 12\,\text{ms}$)
- **Niedrigste Priorität:** τ_3 ($T_3 = 24\,\text{ms}$)

(5 Punkte)

1.3 RMS-Schrankentest: Da die berechnete Systemauslastung $U = \frac{2}{3} \approx 66{,}7\,\%$ strikt kleiner ist als die Liu-und-Layland-Schranke von $\approx 78{,}0\,\%$ für $n = 3$ Tasks, ist das System **garantiert (hinreichend) schedulbar**. Unter den genannten Modellannahmen halten alle Jobs ihre Deadlines. Eine Übertragung auf andere Annahmen benötigt eine passende neue Analyse. *(5 Punkte)*

1.4 Voraussetzungen und Gegenfälle (zwei begründete Paare reichen, je 5 Punkte):

1. Unabhängige Aufgaben: Ein gehaltener Mutex kann zusätzliche Blockierungszeit verursachen.
2. Implizite Deadlines: Bei $D_i < T_i$ reicht der einfache Auslastungstest nicht unverändert.
3. Keine Zusatzkosten: Interrupt- und Wechselarbeit muss im tatsächlichen Modell berücksichtigt werden.
4. Ein Prozessor und vollständige Präemption: Zwei getrennte Kernkontexte oder lange nicht präemptierbare Abschnitte entsprechen diesem Modell nicht.

(10 Punkte)

Aufgabe 2: Synchronisation & Priority Inversion (20 Punkte)

2.1 Erklären Sie den Unterschied zwischen einem CMSIS-RTOS2-Mutex und einem binären Semaphor im verwendeten FreeRTOS-Adapter. Gehen Sie auf *Ownership*, den Anfangswert eines Ereignissemaphors und die zulässige Verwendung aus einer ISR ein. ISR bedeutet Interrupt Service Routine. *(10 Punkte)*

2.2 Beschreiben Sie die indirekte Verzögerung von H bei $H > M > L$, wenn L einen von H benötigten Mutex hält und M ohne Ressourcenzugriff bereit wird. Wie hilft Prioritätsvererbung im Ein-Mutex-Fall? Welche H-Wartezeit bleibt? Beseitigt Vererbung einen Ressourcenwartezyklus? *(10 Punkte)*

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 2

Zu 2.1 (Mutex vs. Semaphor): Ein Mutex besitzt **Ownership** (Besitzrecht). Nur die Task, die den Mutex gesperrt (`Acquire`) hat, darf ihn auch wieder entsperren (`Release`). Er dient primär dem Schutz gemeinsamer Daten (Critical Sections). Wegen des Ownerships darf ein Mutex **niemals** in einer Hardware-ISR verwendet werden.

Ein binärer Semaphor hat kein Ownership. Für ein noch nicht eingetretenes Ereignis startet er mit null. Eine ISR darf bei gültigem Handle, zulässiger IRQ-Priorität und passender API einen Hinweis durch `osSemaphoreRelease` geben. Ein voller binärer Semaphor zählt nicht mehrere Hinweise. Eine Aufweckung macht die Task Ready; die spätere Zuteilung macht sie Running. *(10 Punkte)*

Zu 2.2 (Priority Inversion & PIP): Beteiligt sind eine Low-, eine Medium- und eine High-Prio-Task. Die Low-Task sperrt einen Mutex. Die High-Task möchte denselben Mutex und blockiert. Nun wird die Medium-Task ready, präemptiert die Low-Task (da $P_{\text{Med}} > P_{\text{Low}}$) und rechnet weiter. Die High-Task wartet damit indirekt auf die Medium-Task — eine Prioritätsinversion liegt vor.

Lösung durch PIP: Das Betriebssystem erkennt die Blockade und *vererbt* der Low-Task temporär die Priorität der High-Task (Priority Boost). Die Medium-Task kann die Low-Task nicht mehr präemptieren. Die Low-Task beendet unter diesen Voraussetzungen ihre verbleibende geschützte Arbeit, gibt den Mutex frei und verliert im einfachen Ein-Mutex-Fall die Anhebung. H kann die Ressource erwerben. H wartet weiterhin auf Ls Restarbeit. Mehrere Mutexe oder Gerätewartezeiten brauchen zusätzliche Betrachtung; Vererbung beseitigt keinen Deadlock durch einen Ressourcenzyklus. *(10 Punkte)*

Aufgabe 3: Multicore & Architektur (20 Punkte)

Sie entwerfen ein System auf dem STM32H755 (Cortex-M7 und Cortex-M4).

3.1 Erklären Sie den Unterschied zwischen SMP (*Symmetric Multiprocessing*) und AMP (*Asymmetric Multiprocessing*). Ordnen Sie M7 Bare Metal und M4 mit lokalem FreeRTOS ein. Bietet diese Rollenaufteilung automatisch zeitliche Isolation? Nennen Sie zwei weiterhin gemeinsame Ressourcen. *(10 Punkte)*

3.2 Was ist ein *Cache Coherency Failure*? Beschreiben Sie, wie dieser Fehler auftritt, wenn der M7-Kern über Write-Back-Caches in den geteilten SRAM4 schreibt und der M4-Kern versucht, diese Daten zu lesen. Welche Rolle spielt eine passend konfigurierte nicht cachebare MPU-Region? Welche beiden zusätzlichen Verträge bleiben für geordnete und exklusive Veröffentlichung nötig? *(10 Punkte)*

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 3

Zu 3.1 (SMP vs. AMP): Bei SMP verwaltet ein Betriebssystem mehrere Kerne; Migration ist eine mögliche Konfiguration, keine zwingende Eigenschaft jedes Jobs. AMP beschreibt getrennte Firmware-/Betriebssystemkontexte und kann auch auf gleichen Kernen eingesetzt werden. Die hier benutzte Aufteilung M7 Bare Metal, M4 mit lokalem FreeRTOS ist AMP.

Beide Modelle können mit geeigneter Analyse Echtzeitanforderungen erfüllen. AMP allein garantiert weder WCET noch zeitliche Isolation. Auf dem H755 bleiben beispielsweise gemeinsame Bus-/Speicherwege, Taktverwaltung und Reset. Kernrollen und Ressourcenkonflikte sind getrennt zu prüfen.

(10 Punkte)

Zu 3.2 (Cache-Kohärenz): M7 kann einen neuen Wert zunächst nur in einer Dirty-Cachekopie halten; SRAM enthält noch den alten Wert. M4 liest diesen alten SRAM-Wert. Umgekehrt kann M7 eine alte gültige Kopie lesen, nachdem M4 SRAM geändert hat. `volatile` und DMB bewirken keinen automatischen Cacheabgleich.

Eine tatsächlich wirksame Normal-Non-Cacheable-Region für den kleinen Kanal vermeidet diese Cachekopien; Regionsüberlappung und Startreihenfolge werden geprüft. Sie macht eine mehrteilige Operation nicht unteilbar. Zusätzlich braucht der Kanal einen Besitzvertrag und geordnete Veröffentlichung beziehungsweise Konsum mit den passenden Barrieren. Alternativ sind korrekt isolierte Cachezeilen und ein vollständiger Cachewartungsvertrag möglich. Das Shareable-Bit allein erzeugt auf diesem System keinen automatischen M7/M4-Abgleich.

(10 Punkte)

Aufgabe 4: Safety & Watchdogs (15 Punkte)

4.1 Im Lehrmodell wird ein IWDG ohne Fenster betrieben. Ein WWDG hat $f_{\mathrm{PCLK}} = 100\,\mathrm{MHz}$, Vorteiler $P = 8$, Reload $R = 127$ und Fenster $W = 80$. Erlaubt ist $63 < T \leq 80$, erster Abwärtsschritt ideal nach einem vollen Tick. Berechnen Sie mit $\Delta t = 4096P/f_{\mathrm{PCLK}}$ Tickdauer und Zeit bis Fensteröffnung. Beurteilen Sie einen Refresh nach 5 ms. Beweist ein Refresh im Fenster, dass alle Validierungsroutinen korrekt durchlaufen wurden? *(8 Punkte)*

4.2 Der WWDG fordert EWI bei $T = 64$ an und setzt beim nächsten Schritt nach 63 zurück. Erklären Sie Maskierbarkeit und nominale Restzeit. Im gegebenen Lehrvertrag muss ENABLE bei Fehler und während Reset/Boot inaktiv bleiben; der EWI soll kurz Low setzen und **nicht refreshen**. Warum darf sich der Ausgangsschutz trotzdem nicht allein auf diesen Callback verlassen? Nennen Sie eine Beschaltungs- und eine Bootmaßnahme für einen isolierten aktiven High-Ausgang. *(7 Punkte)*

Ihre Lösung

(Notieren Sie Rechnungen und Begründungen hier.)

Musterlösung Aufgabe 4

4.1: $\Delta t = 4096 \cdot 8/10^8 = 0{,}32768\,\text{ms}$ (2 P). Fensteröffnung nach $127 - 80 = 47$ Schritten: $47 \cdot 0{,}32768 = 15{,}40096\,\text{ms}$ (2 P). Bei 5 ms ist der Zähler noch oberhalb 80, Refresh ist zu früh und löst im Modell Reset aus (2 P). IWDG ohne Fenster erkennt zu frühes Füttern nicht. WWDG prüft den Refreshzeitpunkt, keine funktionale Korrektheit der Validierung; auch fehlerhafte Software kann rechtzeitig füttern (2 P). Ein passend unterstütztes IWDG-Fenster ist eine andere Konfiguration.

4.2: EWI ist ein maskierbarer Interrupt, kein NMI; unter den gegebenen Annahmen bleibt nominal ein Tick, 327,68 µs (2 P). Zu früher Refresh kann ohne EWI resetten, Maskierung oder Verdrängung kann die Bedienung verhindern (2 P). Kurzes Low-Schreiben ohne Refresh entspricht der **gegebenen Policy**, ist keine universelle Hardwarevorschrift (1 P). Geeigneter externer Pull-down hält den aktiven High-Ausgang bei hochohmigem Reset inaktiv (1 P). Boot initialisiert Low und erlaubt Freigabe erst nach den vereinbarten Prüfungen (1 P). Ein Lehrmarker beweist keine Sicherheit einer realen Maschine.

Aufgabe 5: Code-Review (20 Punkte)

Der folgende **absichtlich fehlerhafte C-Ausschnitt** soll jeden angenommenen Arbeitshinweis bearbeiten. Ein gültiger Mutex ist bereits angelegt; Boot und GPIO sind geprüft. `HAL_Delay` ist in diesem Projekt aktives Warten auf die HAL-Millisekundenbasis. `osDelay` verwendet Kernelticks. Aussagen über konkrete Interleavings beziehen sich auf das gezeigte Maschinenmodell; dies ist kein Beispiel für korrekte portable C-Synchronisation.

Benennen Sie **vier unterschiedliche Probleme** und schlagen Sie einen passenden Kommunikations-/Wartevertrag vor. Zählen Sie Mutexverwendung und den zusätzlich blockierenden Parameter desselben ISR-Aufrufs nicht als zwei unabhängige Funde. Muss ein unzulässiger API-Aufruf zwangsläufig einen HardFault erzeugen? *(20 Punkte)*

```c
static volatile uint32_t pending = 0;
osMutexId_t dataMutex; /* gültig angelegt */
void HAL_GPIO_EXTI_Callback(uint16_t pin) {
    osMutexAcquire(dataMutex, osWaitForever); /* Ergebnis ignoriert */
    ++pending;
    osMutexRelease(dataMutex);
}
void ProcessTask(void *argument) {
    for (;;) {
        while (pending == 0) { }
        pending = 0;
        Filter_Complex_Data(); /* bearbeitet einen Hinweis */
        HAL_Delay(50);
    }
}
```

Ihre Lösung

(Benennen Sie Ursache, Folge und eine passende Korrektur.)

Musterlösung Aufgabe 5

1. **Mutex-/Warteaufruf im ISR-Kontext:** CMSIS-Mutexfunktionen sind dort unzulässig; `osWaitForever` darf eine ISR auch nicht blockieren. Der Rückgabewert wird zusätzlich ignoriert. Ein Adapter kann einen Fehler melden oder eine Assertion auslösen; ein HardFault ist keine zwangsläufige Folge. Dies ist zusammen ein Fund. Passend ist eine ISR-zulässige Signalisierung mit zulässiger IRQ-Priorität und Timeout null bei Queue-Put. (5 P)
2. **Busy-Wait in der Task:** Die Abfrageschleife bleibt Ready beziehungsweise Running und verbraucht ihre zugeteilte CPU-Zeit. Niedrigere Tasks können verhungern, höhere weiterhin präemptieren. Stattdessen blockiert der Worker an einem passenden Kernelobjekt bis zum Hinweis. (5 P)
3. **Ungeschützter und verlustbehafteter Hinweisvertrag:** Der Worker benutzt den Mutex nicht. `volatile` macht Inkrement und Löschen nicht unteilbar. Ein IRQ zwischen positiver Prüfung und Löschen kann einen neuen Hinweis verlieren; mehrere gezählte Hinweise werden zudem blind auf null gesetzt, aber nur einmal bearbeitet. Geeignet ist ein begrenzter zählender Semaphor oder eine Wertqueue mit expliziter Kapazitäts-/Überlaufregel. Ein einzelnes Flag genügt nur für den anders definierten Vertrag „mindestens ein Hinweis“. (5 P)
4. **Aktive Wartezeit nach der Verarbeitung:** `HAL_Delay(50)` lässt die Task in dieser Projektdefinition ausführungsbereit und nimmt niedrigeren Tasks Zeit. Ein Kernel-Warteaufruf macht sie Blocked; Millisekunden müssen anhand der gegebenen Tickfrequenz in Ticks umgerechnet werden. Das bloße Ersetzen der Zahl garantiert keine exakten physischen 50 ms. (5 P)

Schwere Verarbeitung im Thread ist grundsätzlich passend. Gültige Handles, Startgrenze, erlaubte ISR-Funktion, IRQ-Priorität, Fehlerbehandlung und Kapazitätsvertrag werden gemeinsam geprüft. Auch eine Wertqueue zählt nicht automatisch alle vor dem Callback zusammengefassten physischen Flanken.